

SNIA SDC: StorageAI 2026 - AiSIO: Orchestrating Storage I/O Across CPUs and Accelerators

개요

Samsung 연구원이 AiSIO(Accelerator-integrated Storage I/O, 또는 ACIO)를 발표한다. NVMe 스토리지 I/O를 CPU와 GPU 가속기 양쪽으로 조율하는 오픈소스 프로젝트다. 강연은 가속기 I/O를 제약하는 세 가지 소프트웨어 병목, 지원하는 세 가지 I/O 모드, 기존 libvm 기반 프로토타입을 대체하는 새 아키텍처, 오픈소스 컴포넌트 생태계, 그리고 Samsung Memory Research Center에 설치된 16-드라이브 Dell 서버의 벤치마크 결과를 다룬다.

다루는 주제

- 세 가지 소프트웨어 과제: (1) 애플리케이션, 라이브러리, 언어 런타임, 유저/커널 경계, OS 스토리지 스택 각 레이어의 오버헤드; (2) 가속기에 도달하기 전 호스트 DRAM을 경유하는 원치 않는 데이터 복사; (3) 스토리지 장치의 PCIe 단일 물리 함수 제한(드라이버 하나만 결합 가능).
- 지원하는 세 가지 I/O 모드: CPU 개시(데이터를 호스트 메모리로), CPU 개시 피어-투-피어(CPU가 주도, 데이터는 가속기 메모리로), 장치 개시 피어-투-피어(가속기가 직접 개시, 데이터를 자신의 메모리로).
- 설계 원칙: 대체가 아닌 상호운용성(interoperability over replacement) — 기존 스택을 확장하되 기존 추상화와 성능을 희생하지 않는다.
- 모든 컴포넌트가 Linux 커널 업스트림에 이미 포함되었거나 업스트림을 목표로 하는 오픈소스 프로젝트다.
- 인용된 관련 연구: Nvidia GDS, BAM(Big Accelerator Memory), Nvidia Scattera, AMD ROCK XIO, libenvm. Nvidia GDS 벤치마크 결과 단일 스레드 소규모 I/O에서 NVMe 장치 한 대도 포화시키지 못했다.
- 초기 프로토타입은 SIOV(다중 물리 함수)를 활용해 커널 스택 드라이버와 ACIO 프로토타입 드라이버를 동시에 단일 드라이브에 결합하고, GPU가 LBA를 직접 로드할 수 있도록 파일 익스텐트 캐시를 구성했다.
- 신규 아키텍처는 libvm과 커스텀 커널 패치를 제거하고, HOMIE(Host Orchestrated Multipath IO Daemon), Extend Access

Library(F_MAP_AP 결과 캐시), UDMA buff import, 단일 함수 장치 폴백용 Ublock 인프라를 주요 컴포넌트로 사용한다.

- F_MAP_AP가 파일 익스텐트 조회에서 XFS 온디스크 포맷 직접 디코딩을 대체한다; EBPf 트레이스를 통한 변경 알림 기능을 계획 중이다.
- 오픈소스 컴포넌트: XNME(AiSIO 프리미티브로 확장된 크로스 플랫폼 I/O 라이브러리), XME Perf(벤치마크 도구), UPC(MMIO 도어벨 모듈), Field Path(파일 수준 벤치마크), CHO(프로비저닝 도구).
- DMA buff 인프라 탭 모듈은 아직 업스트림에 포함되지 않았으며 Linux 커널 커뮤니티 피드백을 기다리고 있다.
- 벤치마크 환경: NVMe 드라이브 16개, 비할당 모드, Dell 서버, Samsung Memory Research Center; SPDK BDEV Perf 기준: CPU 8코어로 6,200만 IOPS.
- 애플리케이션 레이어 최적화(XME Perf): 하드웨어 스레드 3개로 3,200만 IOPS; 드라이버 레이어 최적화(UPCE 툴링): CPU 1개/하드웨어 스레드 2개로 약 5,000만 IOPS.
- CPU 효율 개선: 동일한 6,200만 IOPS를 8코어에서 1.5코어 상당으로 달성.
- CPU 개시 피어-투-피어와 장치 개시 피어-투-피어 모두 호스트 메모리 기준과 동일한 IOPS 상한에 도달; DMA 대상이 호스트 메모리든 GPU 메모리든 NVMe 처리 비용이 동일하다.
- 피어-투-피어 대역폭: 4K I/O 크기에서 NVMe 드라이브 3개가 PCIe Gen 5 GPU 링크를 포화시킨다; 4K에서 CPU 한 개로 GPU 8개를 대역폭 기준으로 포화시킬 수 있다.
- Q&A: 벤치마크 파일은 사전 할당된 고정 크기; 데이터 로더 워크로드(이미지 데이터셋, TikTok 데이터셋 포함, 8 GB 파일)는 Nvidia DALI 프레임워크에서 영감을 받았다.

인물 · 조직 · 제품

인물: - Kristoff (강연 중 F_MAP_AP 접근 방식에 이의를 제기한 청중)

조직: - Samsung - Nvidia - AMD - Dell

제품: - AiSIO (ACIO) - HOMIE (Host Orchestrated Multipath IO Daemon) - Nvidia GDS (GPU Direct Storage) - BAM (Big Accelerator Memory) - Nvidia Scattera - AMD ROCK XIO - SPDK (BDEV Perf, NVME Perf) - XNME - XME Perf - UPC - Field Path - CHO - Extend Access Library - Ublock - libenvm - UPCE - Nvidia DALI

수치 및 데이터

- 드라이브당 3.8백만 IOPS — 비할당 모드(NVME LBA format)

- 합계 6,200만 IOPS — CPU 8코어, NVMe 16개 (SPDK BDEV Perf 기준)
- 코어당 7.7백만 IOPS — 기준 성능(선형 스케일링)
- 6.3백만 IOPS — BDEV Perf, 하드웨어 스레드 1개
- 7.4백만 IOPS — BDEV Perf, 하드웨어 스레드 2개
- 1,600만 IOPS — BDEV Perf, CPU 코어 2개 위 하드웨어 스레드 3개
- 3,200만 IOPS — XME Perf, 하드웨어 스레드 3개 (애플리케이션 레이어 최적화)
- 약 4,000만 IOPS — 드라이버 레이어 최적화(UPCE), 하드웨어 스레드 1개
- 약 5,000만 IOPS — 드라이버 레이어 최적화(UPCE), CPU 1개 / 하드웨어 스레드 2개
- 1.5코어 상당 — 최적화 후 6,200만 IOPS 유지에 필요한 CPU 자원
- NVMe 드라이브 3개 — 4K I/O 크기에서 PCIe Gen 5 GPU 링크 포화에 필요한 수량
- GPU 8개 — 4K I/O에서 CPU 한 개로 포화시킬 수 있는 GPU 수(대역폭 기준)
- 8 GB — 데이터 로더 벤치마크에서 사용된 대형 사전 할당 파일 크기

주요 인용

basically we want to extend the existing software stack and we want to do so in ways where we're not trading off the utilities of having existing abstractions and we don't want to trade off performance either. — unknown

that's the vision. We want to be able to have accelerators as uh uh first class citizens in the um in the storage stack. — unknown

a single thread small IO we cannot saturate even a single NVME device using this. — unknown

we hit the 62 million IOPS here with eight CPU cores. So that's 16 devices. Uh that gets saturated by using eight CPU cores. — unknown

the NVME drive doesn't really care that the DMA address is on your host memory or on your GPU. The cost of processing that IO is the same. — unknown

is a proper way to do it and I told for two years to multiple members in your team how to do it and it's really upsetting that you can keep publishing kind of dangerous — Kristoff

debugging tool. It has different content for different file systems. It's a massive risk for uh cost and corruption due to concurrent activity. That's why I told everyone including your boss two years ago, you need to expand the PFS block layout layouts that have all the mechanisms to deal with that including relocation mechanism mechanism for defending clients — Kristoff